

DL Midsem Notes — Section 7

Variants of Gradient Descent

1 Theory

1.1 Batch, Stochastic, Mini-Batch

Batch GD: use the whole dataset to compute one gradient per update — accurate but slow, one update per epoch. **SGD:** one random example per update — fast, noisy, the noise can actually help escape shallow local minima/saddles, but the path is erratic. **Mini-batch GD:** a middle ground (e.g. 32–256 examples per update) — the standard in practice, balancing gradient-estimate stability against update frequency.

1.2 Momentum

Physical analogy: a ball rolling downhill accumulates velocity instead of reacting only to the current slope.

$$v_t = \beta v_{t-1} - \eta \nabla L(w_t), \quad w_{t+1} = w_t + v_t$$

β (typically ~ 0.9) controls how much past velocity persists. Momentum damps oscillation across narrow valleys (where the gradient keeps flipping sign side-to-side) while accelerating consistently in a shared downhill direction.

1.3 Nesterov Accelerated Gradient (NAG)

Momentum first, gradient second: compute the gradient not at the current point but at the *look-ahead* point $w_t + \beta v_{t-1}$ — a correction based on where momentum is about to carry you anyway:

$$v_t = \beta v_{t-1} - \eta \nabla L(w_t + \beta v_{t-1}), \quad w_{t+1} = w_t + v_t$$

This lets NAG "peek ahead" and slow down before overshooting a minimum, which plain momentum can't do.

1.4 AdaGrad

Give each parameter its own adaptive learning rate, shrinking it for parameters that have historically had large gradients:

$$G_t = G_{t-1} + g_t^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t$$

($g_t = \nabla L(w_t)$, ϵ tiny constant for numerical stability.) **Problem:** G_t only ever grows, so the effective learning rate monotonically shrinks toward 0 over training — learning can grind to a halt long before convergence.

1.5 RMSProp

Fixes AdaGrad's problem by replacing the ever-growing sum with an exponential moving average, so old gradients "decay away" instead of accumulating forever:

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) g_t^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

The effective learning rate can now go back up again if recent gradients are small, unlike AdaGrad.

1.6 Adam

Combines momentum (first moment) with RMSProp (second moment), plus a bias correction for both:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, & v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, & \hat{v}_t &= \frac{v_t}{1 - \beta_2^t}, & w_{t+1} &= w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \end{aligned}$$

Why bias correction: $m_0 = v_0 = 0$, so early estimates are biased toward 0 (they haven't accumulated enough history yet); dividing by $(1 - \beta_1^t)$ and $(1 - \beta_2^t)$ corrects for this early-step bias (derivation in Q4).

2 Practice Questions

Q1 (PYQ-style, CS6510-4 Q2A). Differentiate between SGD, Momentum GD, and NAG.

Q2 (PYQ-style, CS6510-2 Q8 & CS6510-5 Q1c). What problem occurs if the learning rate is too high or too low? Why is an adaptive learning rate based on cumulative squared gradients used in AdaGrad? Write the AdaGrad and RMSProp weight update equations and explain how RMSProp fixes AdaGrad's problem.

Q3 (PYQ-style, CS6510-7 Q2a). Explain the Adam optimization algorithm, its key components, and how it differs from standard SGD. What is the advantage of using momentum (first moment estimate) in gradient-based optimization?

Q4 (PYQ-style, CS6510-5 Q3b). Show why Adam's bias correction naturally disappears as the number of steps used to compute the exponential moving averages gets large.

Q5 (PYQ-style, CS6510-5 Q2a, adapted — true/false, correct if false).

- (a) Simple models have high bias and high variance.
- (b) High bias and low variance are the desirable characteristics of a good model.
- (c) Variance contributes to error while bias does not contribute to error.
- (d) In the NAG optimizer, the gradient term is first added to the old weight, and then the momentum term is added.

3 Answers

A1.

- **SGD**: update using the gradient from a single (random) example; noisy, erratic path, but cheap per step and the noise helps escape shallow traps.
- **Momentum GD**: accumulates an exponentially-weighted running velocity from past gradients, updates using that velocity — smooths the noisy path and accelerates in a consistent direction.
- **NAG**: like Momentum, but the gradient used to update velocity is evaluated at the look-ahead point $w_t + \beta v_{t-1}$, not at w_t itself — this "look before you leap" correction reduces overshoot near minima that plain momentum is prone to.

A2. Too high η : overshoot / divergence / oscillation across the loss surface. Too low η : painfully slow convergence, can stall on plateaus. AdaGrad uses cumulative squared gradients so that parameters which have received large/frequent gradients get a shrunk effective step (avoiding overshoot there), while rarely-updated parameters keep a relatively larger step — a per-parameter adaptive rate instead of one global η .

$$\text{AdaGrad: } G_t = G_{t-1} + g_t^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t$$

$$\text{RMSProp: } E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)g_t^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

AdaGrad's G_t only accumulates (never forgets), so the effective step size shrinks monotonically toward 0 and learning can halt prematurely. RMSProp replaces the running *sum* with a running *exponential average*, so old, stale gradient information decays away — the effective step size can rise again if recent gradients shrink, instead of being permanently killed by gradients from early training.

A3. Adam maintains two exponential moving averages per parameter: m_t (first moment — mean of recent gradients, i.e. momentum) and v_t (second moment — mean of recent squared gradients, i.e. RMSProp's adaptive scaling), both bias-corrected, combined as

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Unlike plain SGD (which uses only the current gradient, one fixed global η), Adam uses a smoothed, adaptively-scaled direction that is both momentum-accelerated and per-parameter-rescaled. **Advantage of momentum specifically**: it damps the noisy, high-variance updates SGD produces (especially in narrow curved valleys where the raw gradient direction oscillates), and it lets the optimizer keep moving through small local bumps/saddle regions using accumulated velocity rather than getting stuck the instant the instantaneous gradient shrinks.

A4. Consider m_t initialized at $m_0 = 0$ and updated as $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$. Expanding the recursion:

$$m_t = (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} g_i$$

If the true gradient were constant, $g_i = g$, then $m_t = (1 - \beta_1)g \sum_{i=1}^t \beta_1^{t-i} = (1 - \beta_1)g \cdot \frac{1 - \beta_1^t}{1 - \beta_1} = g(1 - \beta_1^t)$. So $\mathbb{E}[m_t] = g(1 - \beta_1^t)$, i.e. a biased (shrunk) estimate of the true gradient by a factor

$(1 - \beta_1^t)$ — this bias is worst at small t (since $m_0 = 0$ pulls early averages toward 0). Dividing by exactly this factor,

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

recovers an unbiased estimate. As $t \rightarrow \infty$, $\beta_1^t \rightarrow 0$ (since $0 < \beta_1 < 1$), so $(1 - \beta_1^t) \rightarrow 1$ and $\hat{m}_t \rightarrow m_t$ — the correction factor becomes 1 and naturally disappears; it only mattered in early steps. The identical argument applies to v_t with β_2 .

A5. (a) **False.** Simple models have *high bias* but *low* variance (they underfit consistently rather than varying wildly across datasets). (b) **False.** The desirable combination is *low* bias and low variance; high bias alone (even with low variance) means the model is consistently wrong (underfitting). (c) **False.** Both contribute: $\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible noise}$. (d) **False.** In NAG, the momentum term is added *first* to get the look-ahead position, and the gradient is then computed and added at that look-ahead point — not the other way around.